

T3 Redes neuronales 1

Índice

1. Introducción
2. Perceptrón: el problema XOR
3. Perceptrones multicapa
4. Funciones de activación
5. TensorFlow Playground
6. Ejemplo de MLP para clasificación
7. Ejemplo de MLP para regresión
8. Consideraciones finales

1 Introducción

Objetivo: introducir los perceptrones multicapa con ejemplos de uso

El problema XOR: problema simple que no puede resolverse con un modelo lineal clásico (Perceptrón)

Perceptrones multicapa: generalización del Perceptrón clásico que constituye la base de las redes actuales

TensorFlow Playground: herramienta web para jugar con redes sencillas aplicadas a datos 2d

Ejemplo de MLP para clasificación: tarea MNIST

Ejemplo de MLP para regresión: tarea auto-mpg

Consideraciones finales: antecedentes, importancia de la profundidad, etc.

2 Perceptrón: el problema XOR

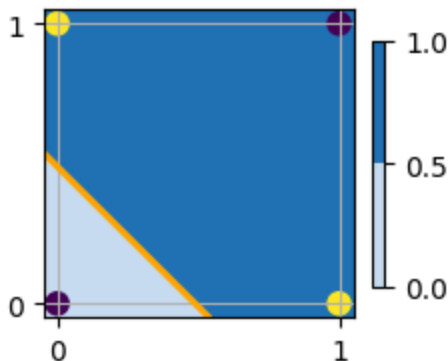
Perceptrón: versión determinista de regresión logística binaria

$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbb{I}(\mathbf{w}^t \mathbf{x} + b \geq 0) = H(\mathbf{w}^t \mathbf{x} + b)$$

Libro Perceptrons: publicado en 1969 por Marvin Minsky y Seymour Papert, proporcionó numerosos ejemplos de problemas de clasificación que los perceptrones no resuelven

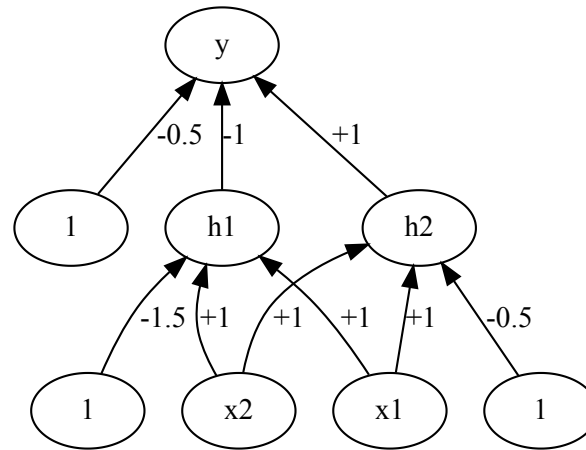
Problema XOR: problema famoso del libro Perceptrons con datos 2d no linealmente separables

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([0, 1, 1, 0], dtype=int); # print(np.c_[X, y])
w = np.array([1, 1]); b = -0.5; P = lambda x: np.heaviside(w @ x + b, 0.0)
fig, ax = plt.subplots(1, 1, figsize=(2.5, 2)); ax.grid()
ticks = [.0, 1.]; ax.set_xticks(ticks); ax.set_yticks(ticks)
R = np.linspace(-.05, 1.05, 50); x1, x2 = np.meshgrid(R, R)
XX = np.c_[np.ravel(x1), np.ravel(x2)]; yy = np.apply_along_axis(P, 1, XX)
ax.contour(x1, x2, yy.reshape(x1.shape), 1, colors='orange', linestyle='solid')
cp = ax.contourf(x1, x2, yy.reshape(x1.shape), 1, cmap='Blues')
plt.colorbar(cp, ax=ax, shrink=0.8); ax.scatter(*X.T, c=y, s=64);
```



Resolución de XOR con una composición de tres Perceptrones: dos unidades ocultas, h_1 y h_2 , y la salida y

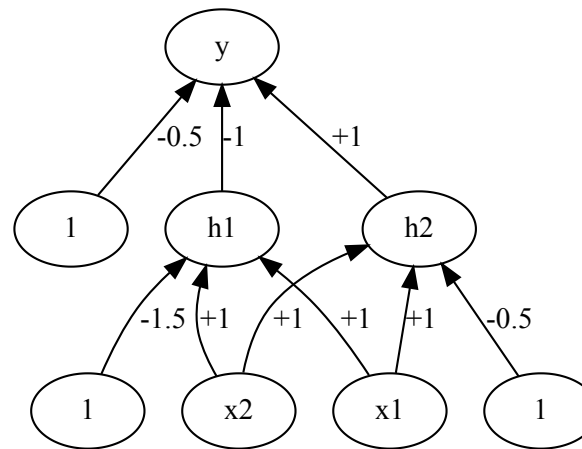
```
In [ ]: import graphviz; graphviz.Source('''digraph { rankdir=BT
yb [label="1"] yb -> y [label="-0.5"] h1 -> y [label="-1"] h2 -> y [label="+1"]
h1b [label="1"] h1b -> h1 [label="-1.5"] x1 -> h1 [label="+1"] x2 -> h1 [label="+1"]
h2b [label="1"] h2b -> h2 [label="-0.5"] x1 -> h2 [label="+1"] x2 -> h2 [label="+1"]
}''').render(filename='MLPXOR', format='svg');
```



x_1	x_2	$w_1^t x + b_1$	h_1	$w_2^t x + b_2$	h_2	$w_y^t h + b_y$	y
0	0	-1.5	0	-0.5	0	-0.5	0
0	1	-0.5	0	+0.5	1	+0.5	1
1	0	-0.5	0	+0.5	1	+0.5	1
1	1	+0.5	1	+1.5	1	-0.5	0

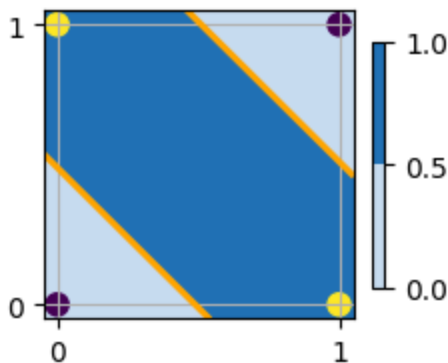
$$y = \neg h_1 \wedge h_2 = \neg(x_1 \wedge x_2) \wedge (x_1 \vee x_2)$$

Resolución de XOR con una composición de tres Perceptrones (cont.):



```

In [ ]: w1 = np.array([1, 1]); b1 = -1.5; w2 = np.array([1, 1]); b2 = -0.5; wy = np.array([-1, 1]); by = -0.5
P = lambda x, w, b: np.heaviside(w @ x + b, 0.0)
MLPXOR = lambda x: P(np.array([P(x, w1, b1), P(x, w2, b2)]), wy, by)
fig, ax = plt.subplots(1, 1, figsize=(2.5, 2)); ax.grid()
ticks = [.0, 1.]; ax.set_xticks(ticks); ax.set_yticks(ticks)
R = np.linspace(-.05, 1.05, 50); x1, x2 = np.meshgrid(R, R)
XX = np.c_[np.ravel(x1), np.ravel(x2)]; yy = np.apply_along_axis(MLPXOR, 1, XX)
ax.contour(x1, x2, yy.reshape(x1.shape), 1, colors='orange', linestyle='solid')
cp = ax.contourf(x1, x2, yy.reshape(x1.shape), 1, cmap='Blues')
plt.colorbar(cp, ax=ax, shrink=0.8); ax.scatter(*X.T, c=y, s=64);
    
```



3 Perceptrones multicapa

Multilayer perceptron (MLP): composición de L funciones o **capas** $f^{(\ell)}(\mathbf{x}^{(\ell)}; \boldsymbol{\theta}^{(\ell)})$

$$f(\mathbf{x}; \boldsymbol{\theta}) = f^{(L)}(f^{(L-1)}(\dots f^{(1)}(\mathbf{x}) \dots)) \quad \text{con} \quad \mathbf{x}^{(\ell+1)} = f^{(\ell)}(\mathbf{x}^{(\ell)}) = \boldsymbol{\varphi}^{(\ell)}(\mathbf{a}^{(\ell)}) \quad \text{y} \quad \mathbf{a}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{x}^{(\ell)} + \mathbf{b}^{(\ell)}$$

Preactivación de la capa ℓ , $\mathbf{a}_\ell$: afinidad de la salida de la capa anterior, $\mathbf{x}^{(\ell)}$; con $\mathbf{x}^{(1)} = \mathbf{x}$

Activación/salida de la capa ℓ , $\mathbf{x}^{(\ell+1)}$: transformación (elemental) de $\mathbf{a}^{(\ell)}$ con $\varphi : \mathbb{R} \rightarrow \mathbb{R}$ **diferenciable** (y no lineal)

Terminología usual:

- "Capa" de entrada: $\mathbf{x}^{(1)} = \mathbf{x}$
- **Capas ocultas:** $\mathbf{x}^{(\ell+1)}$, $1 \leq \ell < L$, cuya dimensión indica el número de salidas o **unidades ocultas** de la capa
- **Capa de salida:** $\mathbf{x}^{(L+1)}$, típicamente con activación sigmoide o softmax para clasificación; identidad para regresión

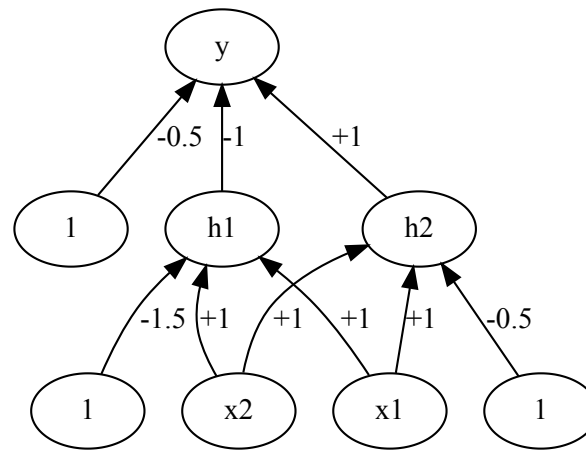
Redes superficiales (sin capas ocultas): si $L = 1$, se reducen a modelos lineales convencionales

- **Regresión logística binaria:** $f(\mathbf{x}; \boldsymbol{\theta}) = x^{(2)}$, $x^{(2)} = \sigma(a^{(1)})$, $a^{(1)} = \mathbf{W}^{(1)} \mathbf{x} + b^{(1)}$
- **Regresión logística:** $f(\mathbf{x}; \boldsymbol{\theta}) = x^{(2)}$, $x^{(2)} = \mathcal{S}(a^{(1)})$, $a^{(1)} = \mathbf{W}^{(1)} \mathbf{x} + b^{(1)}$
- **Regresión lineal:** $f(\mathbf{x}; \boldsymbol{\theta}) = x^{(2)}$, $x^{(2)} = a^{(1)}$, $a^{(1)} = \mathbf{W}^{(1)} \mathbf{x} + b^{(1)}$

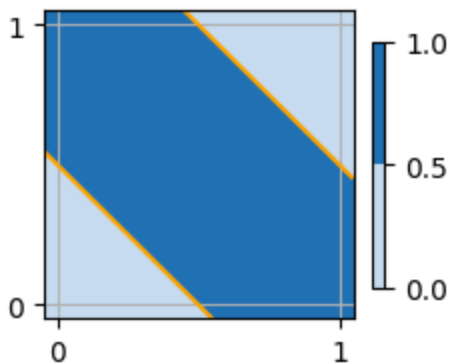
Redes profundas (con capas ocultas): si $L > 1$, son modelos lineales con **ingeniería de características** integrada

Ejercicio: sustituye las Heavisides del MLP para XOR por $\sigma(z) = \frac{1}{1 + e^{-Tz}}$, $T = 10$

Solución: MLP para XOR con sigmoides atemperadas



```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; from scipy.special import expit
w1 = np.array([1, 1]); b1 = -1.5; w2 = np.array([1, 1]); b2 = -0.5; wy = np.array([-1, 1]); by = -0.5
P = lambda x, w, b: expit(10 * (w @ x + b)); MLPXOR = lambda x: P(np.array([P(x, w1, b1), P(x, w2, b2)]), wy, by)
fig, ax = plt.subplots(1, 1, figsize=(2.5, 2)); ax.grid()
ticks = [.0, 1.]; ax.set_xticks(ticks); ax.set_yticks(ticks)
R = np.linspace(-.05, 1.05, 50); x1, x2 = np.meshgrid(R, R)
XX = np.c_[np.ravel(x1), np.ravel(x2)]; yy = np.apply_along_axis(MLPXOR, 1, XX)
ax.contour(x1, x2, yy.reshape(x1.shape), 1, colors='orange', linestyle='solid')
cp = ax.contourf(x1, x2, yy.reshape(x1.shape), 1, cmap='Blues')
plt.colorbar(cp, ax=ax, shrink=0.8); # ax.scatter(*X.T, c=y, s=64);
```



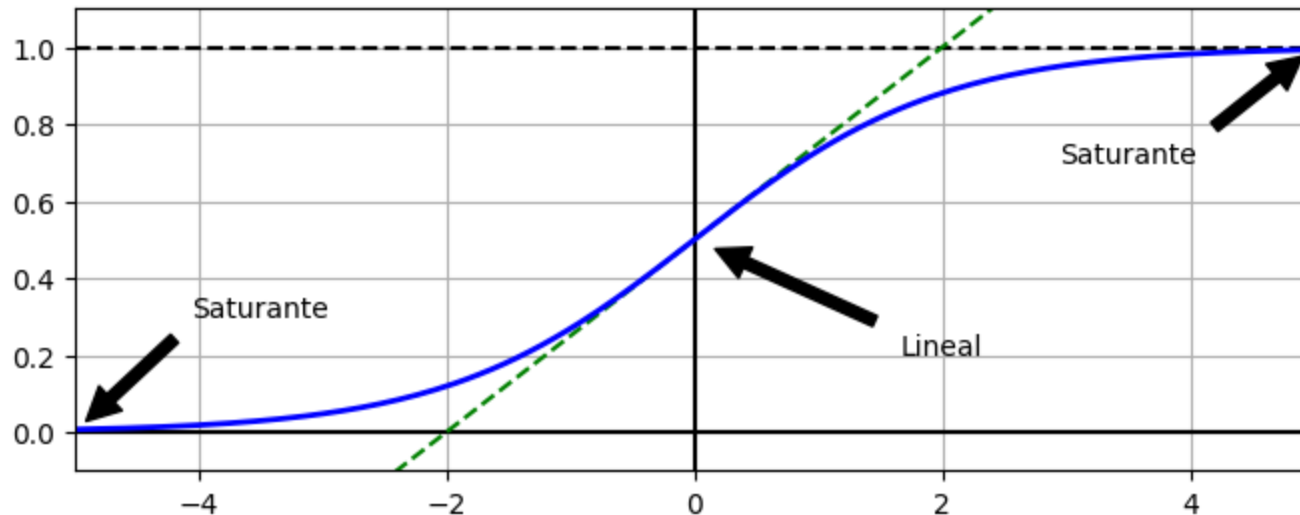
4 Funciones de activación

Necesidad de activaciones no lineales: si $\varphi^{(\ell)}(\mathbf{a}^{(\ell)}) = c^{(\ell)} \mathbf{a}^{(\ell)}$, la red se reduce a un modelo lineal

$$\mathbf{a}^{(L)} = \mathbf{W}^{(L)}(c^{(L-1)} \mathbf{W}^{(L-1)}(\dots(c^{(1)} \mathbf{W}^{(1)} \mathbf{x}) \dots)) = \tilde{\mathbf{W}} \mathbf{x} \quad \text{con} \quad \tilde{\mathbf{W}} = \mathbf{W}^{(L)}(c^{(L-1)} \mathbf{W}^{(L-1)} \dots c^{(1)} \mathbf{W}^{(1)})$$

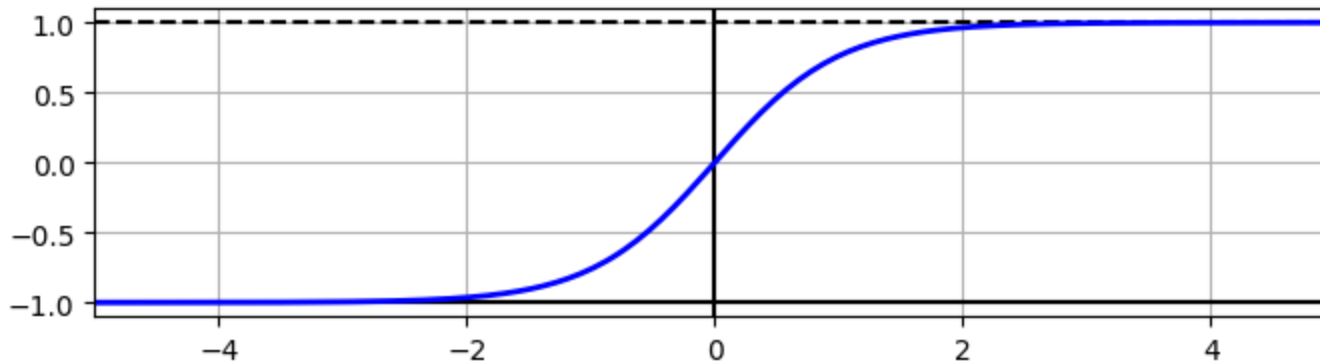
Sigmoide: muy usada en el pasado, satura a (0, 1)

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; a = np.linspace(-5, 5, 200)
plt.figure(figsize=(8,3)); plt.grid(True); plt.axis([-5, 5, -0.1, 1.1]);
plt.plot([-5, 5], [0, 0], 'k-'); plt.plot([-5, 5], [1, 1], 'k--'); plt.plot([0, 0], [-0.1, 1.1], 'k-')
plt.plot([-5, 5], [-3/4, 7/4], 'g--'); plt.plot(a, 1 / (1 + np.exp(-a)), "b-", linewidth=2);
props = dict(facecolor='black', shrink=0.1)
plt.annotate('Saturante', xytext=(3.5, 0.7), xy=(5, 1), arrowprops=props, ha="center")
plt.annotate('Saturante', xytext=(-3.5, 0.3), xy=(-5, 0), arrowprops=props, ha="center")
plt.annotate('Lineal', xytext=(2, 0.2), xy=(0, 0.5), arrowprops=props, ha="center");
```



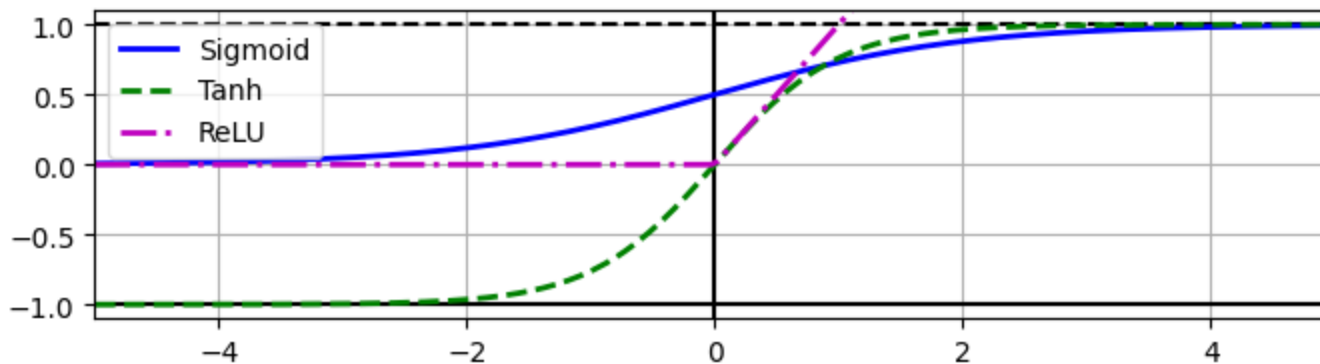
Tangente hiperbólica: muy usada en el pasado, satura a $(-1, 1)$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; a = np.linspace(-5, 5, 200)
plt.figure(figsize=(8,2)); plt.grid(True); plt.axis([-5, 5, -1.1, 1.1]);
plt.plot([-5, 5], [-1, -1], 'k-'); plt.plot([-5, 5], [1, 1], 'k--'); plt.plot([0, 0], [-1.1, 1.1], 'k-')
plt.plot(a, np.tanh(a), "b-", linewidth=2);
```



Rectified linear unit (ReLU): muy usada actualmente, $\text{ReLU}(a) = \max(a, 0) = a \mathbb{I}(a > 0)$

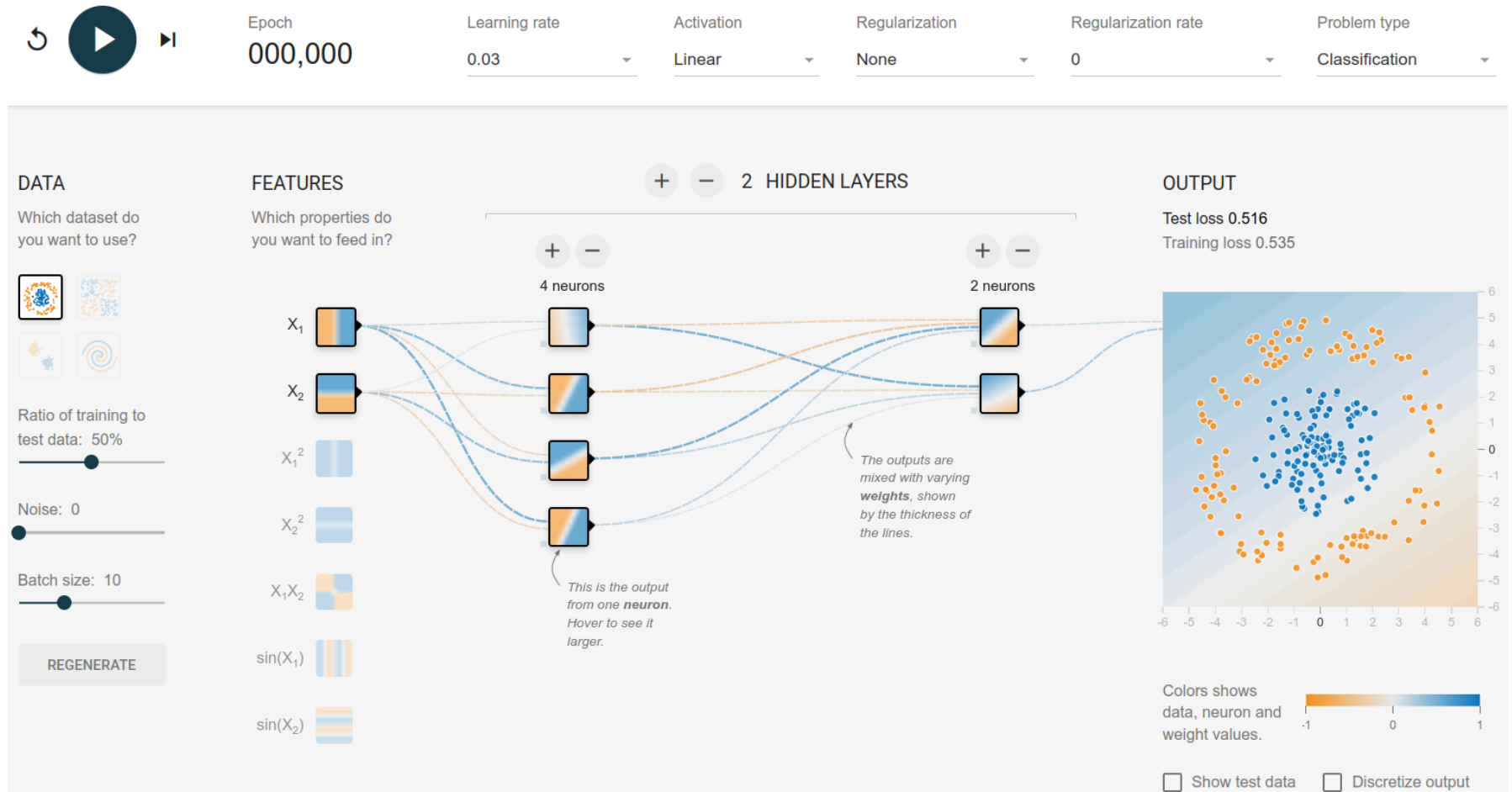
```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; a = np.linspace(-5, 5, 200)
plt.figure(figsize=(8,2)); plt.grid(True); plt.axis([-5, 5, -1.1, 1.1]);
plt.plot([-5, 5], [-1, -1], 'k-'); plt.plot([-5, 5], [1, 1], 'k--'); plt.plot([0, 0], [-1.1, 1.1], 'k-')
plt.plot(a, 1 / (1 + np.exp(-a)), "b-", linewidth=2, label="Sigmoid")
plt.plot(a, np.tanh(a), "g--", linewidth=2, label="Tanh")
plt.plot(a, np.maximum(a, 0), "m-.", linewidth=2, label="ReLU"); plt.legend(loc="upper left");
```



5 TensorFlow Playground

TensorFlow Playground: herramienta web para jugar con redes sencillas aplicadas a datos 2d

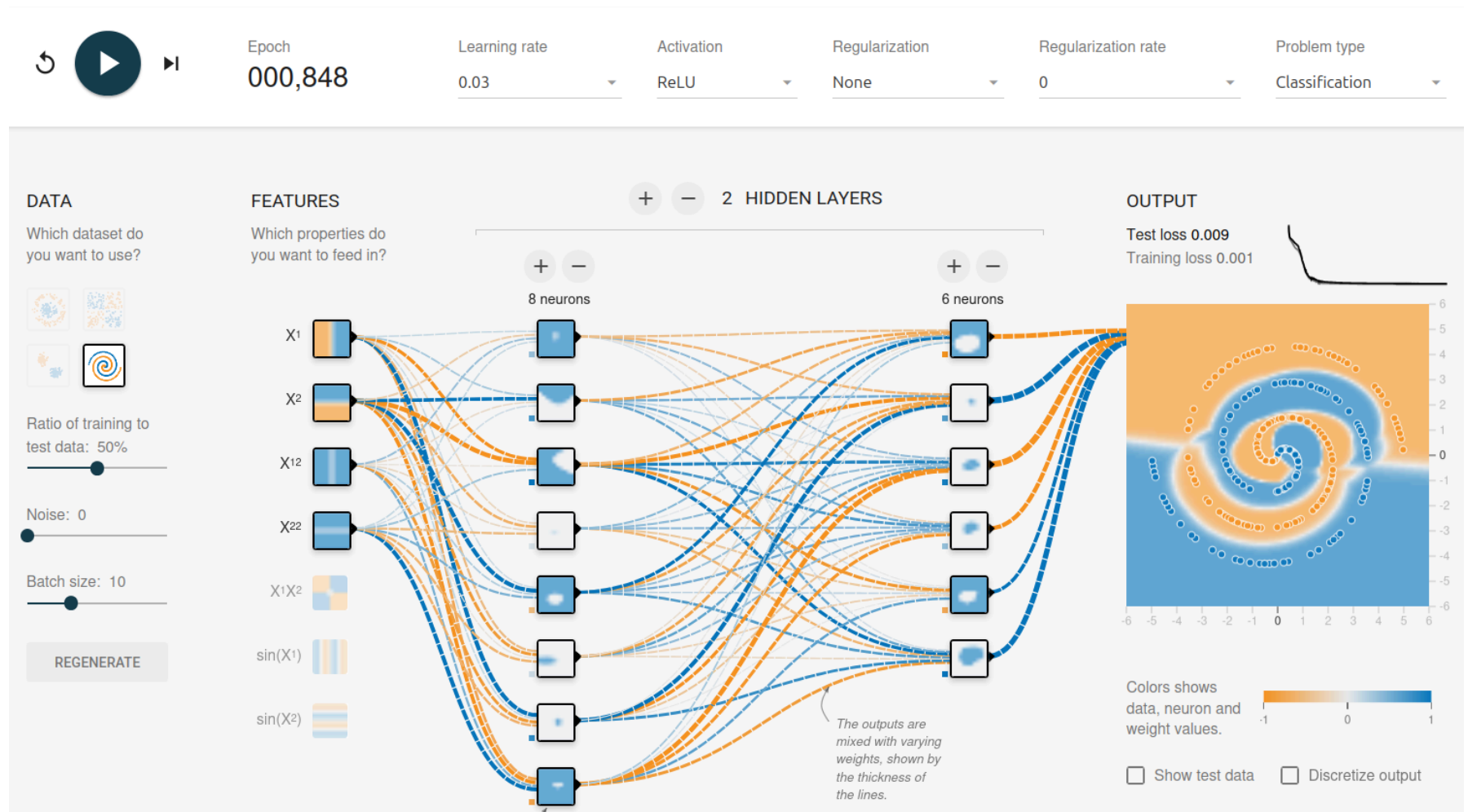
URL y github: <http://playground.tensorflow.org> <https://github.com/tensorflow/playground>



Ejercicio: por cada tarea de clasificación, halla una red minimalista que la resuelva

Solución:

- La lineal no requiere capas ocultas
- La círculo-anillo tampoco con características cuadráticas; eso, o una capa oculta de tres unidades
- La de cuadrantes se resuelve con una capa oculta de cuatro unidades
- La de espirales puede resolverse con características cuadráticas y dos capas ocultas de ocho y seis unidades



6 Ejemplo de MLP para clasificación

MLP para MNIST: entrada de 784 dimensiones, capa oculta de 800 ReLUs y salida softmax para 10 clases

```
In [ ]: import silence_tensorflow; import keras; keras.utils.set_random_seed(23)
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train = x_train.reshape(-1, 28*28).astype("float32") / 255.0
x_test = x_test.reshape(-1, 28*28).astype("float32") / 255.0
M = keras.Sequential([keras.Input(shape=(28*28,)),
    keras.layers.Dense(800, activation='relu'),
    keras.layers.Dense(10, activation='softmax')])
M.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 800)	628,000
dense_1 (Dense)	(None, 10)	8,010

Total params: 636,010 (2.43 MB)

Trainable params: 636,010 (2.43 MB)

Non-trainable params: 0 (0.00 B)

```
In [ ]: M.compile(loss='crossentropy', optimizer='adam', metrics=['accuracy'])
M.fit(x_train, y_train, epochs=5, verbose=0)
loss, acc = M.evaluate(x_test, y_test, verbose=0)
print(f'Loss: {loss:.4f} Precisión: {acc:.2%}')
```

Loss: 0.0777 Precisión: 97.64%

7 Ejemplo de MLP para regresión

Auto-mpg: predicción de consumo en miles per gallon; <https://archive.ics.uci.edu/static/public/9/auto+mpg.zip>

```
In [ ]: import numpy as np; import pandas as pd; pd.set_option('display.max_rows', 4)
names = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'model_year', 'origin', 'car_name']
data = pd.read_csv('auto-mpg.data', sep=r'\s+', header=None, names=names, na_values={'horsepower': '?'})
data.iloc[[0, -1]]
```

```
Out[ ]:
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	car_name
0	18.0	8	307.0	130.0	3504.0	12.0	70	1	chevrolet chevelle malibu
397	31.0	4	119.0	82.0	2720.0	19.4	82	1	chevy s-10

Preproceso preliminar:

- No usamos `car_name` para predecir
- Eliminamos seis casos sin horsepower
- Ponemos nombre a las categorías de `origin` para facilitar su interpretación
- Normalizamos tipos de datos

```
In [ ]: data.drop('car_name', axis=1, inplace=True)
data.dropna(inplace=True)
data['origin'] = data['origin'].map({1: 'USA', 2: 'Europe', 3: 'Japan'}); data.iloc[[0, -1]]
data = data.astype({'cylinders' : np.float64, 'model_year' : np.float64, 'origin' : 'category'})
data.iloc[[0, -1]]
```

```
Out[ ]:
```

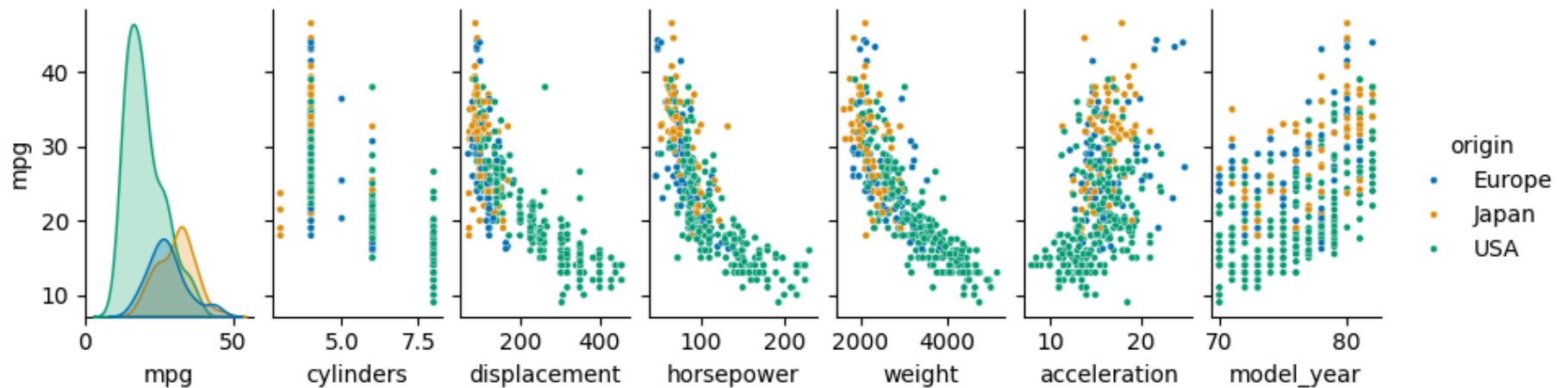
	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin
0	18.0	8.0	307.0	130.0	3504.0	12.0	70.0	USA
397	31.0	4.0	119.0	82.0	2720.0	19.4	82.0	USA

Resumen e inspección visual:

```
In [ ]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
Index: 392 entries, 0 to 397  
Data columns (total 8 columns):  
#   Column          Non-Null Count  Dtype    
---  ---            
0    mpg            392 non-null    float64  
1    cylinders       392 non-null    float64  
2    displacement    392 non-null    float64  
3    horsepower      392 non-null    float64  
4    weight          392 non-null    float64  
5    acceleration    392 non-null    float64  
6    model_year      392 non-null    float64  
7    origin          392 non-null    category  
dtypes: category(1), float64(7)  
memory usage: 25.0 KB
```

```
In [ ]: import seaborn as sns  
sns.pairplot(data, y_vars='mpg', hue="origin", aspect=.5, palette='colorblind', plot_kws=dict(marker="o", s=10));
```



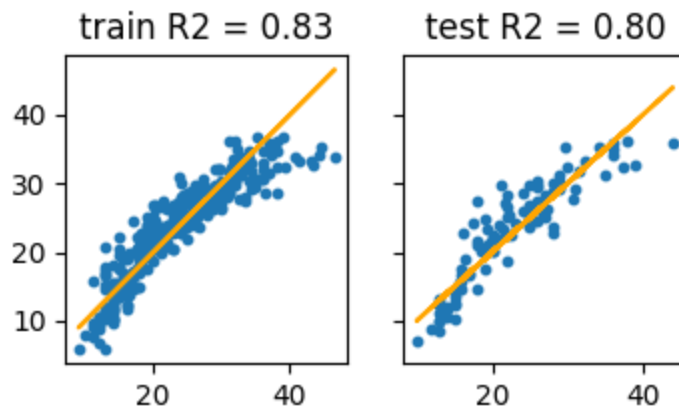
Diseño experimental:

- Binarización de `origin` para facilitar su tratamiento
- Partición de los datos en entrenamiento y test
- Estandarización de características

```
In [ ]: from sklearn.preprocessing import StandardScaler; from sklearn.model_selection import train_test_split
data = pd.concat([data, pd.get_dummies(data.origin, prefix='', prefix_sep='')], axis=1)
data.drop('origin', axis=1, inplace=True); y = data.pop('mpg'); X = data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
scaler = StandardScaler(); X_train = scaler.fit_transform(X_train); X_test = scaler.transform(X_test);
```

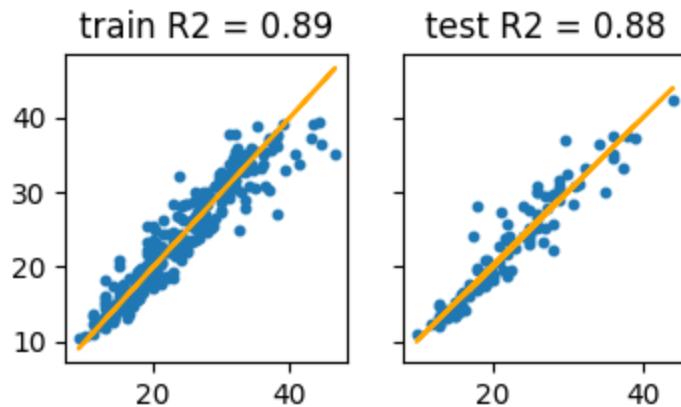
Experimento de referencia con un modelo sencillo: regresión lineal sin ingeniería de características

```
In [ ]: import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression; from sklearn.metrics import r2_score
linreg = LinearRegression().fit(X_train, y_train)
y_train_pred = linreg.predict(X_train); y_test_pred = linreg.predict(X_test)
r2_train = r2_score(y_train, y_train_pred); r2_test = r2_score(y_test, y_test_pred)
fig, axs = plt.subplots(1, 2, figsize=(4, 2), sharey='all')
ax = axs[0]; ax.set_title(f'train R2 = {r2_train:.2f}')
ax.scatter(y_train, y_train_pred, s=10); ax.plot(y_train, y_train, color='orange')
ax = axs[1]; ax.set_title(f'test R2 = {r2_test:.2f}')
ax.scatter(y_test, y_test_pred, s=10); ax.plot(y_test, y_test, color='orange');
```



Experimento con un MLP: dos capas ocultas de 64 ReLUs y capa de salida con una unidad lineal

```
In [ ]: import keras
mlp = keras.Sequential([
    keras.Input(shape=[X_train.shape[1]]),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(1)])
mlp.compile(loss='mean_squared_error', optimizer='adam')
mlp.fit(X_train, y_train, epochs=100, verbose=0)
y_train_pred = mlp.predict(X_train, verbose=0); y_test_pred = mlp.predict(X_test, verbose=0)
r2_train = r2_score(y_train, y_train_pred); r2_test = r2_score(y_test, y_test_pred)
fig, axs = plt.subplots(1, 2, figsize=(4, 2), sharey = 'all')
ax = axs[0]; ax.set_title(f'train R2 = {r2_train:.2f}')
ax.scatter(y_train, y_train_pred, s=10); ax.plot(y_train, y_train, color='orange')
ax = axs[1]; ax.set_title(f'test R2 = {r2_test:.2f}')
ax.scatter(y_test, y_test_pred, s=10); ax.plot(y_test, y_test, color='orange');
```



8 Consideraciones finales

Antecedentes de las redes: las ideas básicas tras las redes son conocidas durante varias décadas (desde la cibernética)

Conexiones con la biología: muchos autores se han inspirado en la biología

Gran popularidad a partir de los 2010: por la mejora de resultados en reconocimiento automático del habla (2011) y clasificación de imágenes (2012)

GPUs y grandes conjuntos de datos supervisados: son los principales factores tras la mejora de resultados

Librerías de código abierto: facilitan la aplicación de las redes a multitud de problemas

- **Tensorflow (Google):** <https://www.tensorflow.org/>
- **PyTorch (Meta):** <https://pytorch.org/>

No necesidad de más de una capa oculta: un MLP de **una sola capa oculta** con suficientes unidades es un **aproximador de funciones universal** ya que puede aproximar toda función (mínimamente suave) tanto como se quiera

Importancia de la profundidad: las redes profundas suelen ofrecer mejores resultados ya que las últimas capas aprovechan las características aprendidas en las primeras capas